

Python: bucles, rangos e iteración

1. Qué significa iterar

Iterar significa repetir una acción varias veces. En programación, una iteración es cada repetición de un bloque de instrucciones. Los bucles permiten automatizar tareas repetitivas: recorrer una lista de notas, mostrar varios mensajes, sumar importes, buscar un dato o procesar los elementos de un diccionario.

Sin bucles, tendríamos que escribir muchas instrucciones repetidas. Con bucles, una misma estructura permite resolver tareas de forma más clara y flexible.

```
print("Intento 1")
print("Intento 2")
print("Intento 3")

# Con un bucle:
contador = 1
while contador <= 3:
    print("Intento", contador)
    contador = contador + 1
```

Idea central: un bucle repite un bloque de código mientras exista algo que recorrer o mientras se cumpla una condición.

2. El bucle while

El bucle while repite un bloque de instrucciones mientras una condición sea verdadera. Se utiliza cuando no sabemos necesariamente cuántas repeticiones harán falta, pero sí conocemos la condición que debe cumplirse para seguir repitiendo.

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

En este ejemplo, el bucle continúa mientras contador sea menor o igual que 5. En cada vuelta se muestra el valor actual y después el contador aumenta en 1.

Parte	Significado
while	Indica que empieza un bucle condicionado.
contador <= 5	Condición que se evalúa antes de cada vuelta.
:	Marca el inicio del bloque indentado.
contador = contador + 1	Actualiza la variable de control.

3. Evitar bucles infinitos

Un bucle infinito ocurre cuando la condición de un while nunca deja de ser verdadera. Es uno de los errores más habituales al empezar a programar con bucles.

```
contador = 1

while contador <= 5:
    print(contador)
    # Falta actualizar contador.
    # El bucle no terminaría nunca.
```

Para evitarlo, normalmente el bloque del while debe modificar alguna variable que afecte a la condición.

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

Antes de escribir un while conviene preguntarse: ¿qué condición lo mantiene activo y qué instrucción hará que esa condición deje de cumplirse?

4. El bucle for

El bucle for se utiliza para recorrer una secuencia de elementos. Esa secuencia puede ser una lista, una tupla, un texto, un diccionario, un conjunto o un rango de números.

```
nombres = ["Ana", "Luis", "Marta"]

for nombre in nombres:
    print(nombre)
```

En cada vuelta del bucle, la variable nombre toma un valor diferente de la lista. Primero vale Ana, después Luis y finalmente Marta.

Parte	Significado
for	Indica que empieza un bucle de recorrido.
nombre	Variable que cambia en cada vuelta.
in	Indica de dónde salen los valores.
nombres	Secuencia que se recorre.
:	Marca el inicio del bloque indentado.

5. Recorrer listas, tuplas y textos

Las listas y las tuplas se recorren de forma parecida: el bucle pasa por cada elemento, de izquierda a derecha.

```
notas = [7, 8, 6, 9]

for nota in notas:
    print("Nota:", nota)
```

También se puede recorrer un texto. En ese caso, Python procesa carácter por carácter.

```
palabra = "Python"

for letra in palabra:
    print(letra)
```

Un objeto que puede recorrerse con un for se llama iterable. Listas, tuplas, strings, diccionarios, co

6. range: generar secuencias numéricas

range sirve para generar una secuencia de números. Se usa mucho con for cuando queremos repetir algo un número concreto de veces.

```
for numero in range(5):
    print(numero)
```

Este código muestra 0, 1, 2, 3 y 4. El 5 no se incluye.

Forma	Qué genera
range(5)	0, 1, 2, 3, 4
range(1, 6)	1, 2, 3, 4, 5
range(2, 11, 2)	2, 4, 6, 8, 10
range(10, 0, -1)	10, 9, 8, 7, 6, 5, 4, 3, 2, 1

7. El inicio, el final y el paso en range

range puede recibir uno, dos o tres valores.

Uso	Estructura	Explicación
Un valor	range(fin)	Empieza en 0 y termina antes de fin.
Dos valores	range(inicio, fin)	Empieza en inicio y termina antes de fin.
Tres valores	range(inicio, fin, paso)	Avanza o retrocede según el paso.

```
for numero in range(1, 10, 2):
    print(numero)
```

```
# Salida:
# 1
# 3
# 5
# 7
# 9
```

En Python, el valor final de range no se incluye. Esta regla es fundamental para evitar errores.

8. Diferencias entre while y for

Bucle	Cuándo usarlo	Ejemplo
while	Cuando el bucle depende de una condición.	Repetir mientras queden intentos.
for	Cuando sabemos qué secuencia queremos recorrer.	Recorrer una lista de alumnos.
for con range	Cuando queremos repetir algo un número conocido de veces.	Mostrar los números del 1 al 10.

```
# while: depende de una condición
saldo = 10
while saldo > 0:
    saldo = saldo - 3
    print("Saldo:", saldo)

# for: recorre una secuencia
for intento in range(1, 4):
    print("Intento", intento)
```

Regla práctica: usa for para recorrer; usa while cuando la repetición depende de una condición que puede cambiar.

9. Contadores y acumuladores

Dos patrones muy frecuentes en bucles son el contador y el acumulador. Un contador cuenta cuántas veces ocurre algo. Un acumulador va sumando valores.

```
notas = [7, 4, 8, 5, 3]

aprobados = 0

for nota in notas:
    if nota >= 5:
        aprobados = aprobados + 1

print("Aprobados:", aprobados)
```

```
precios = [10, 25, 5]

total = 0

for precio in precios:
    total = total + precio

print("Total:", total)
```

10. Recorrer diccionarios

Un diccionario se puede recorrer de varias formas: recorriendo sus claves, sus valores o sus pares clave-valor.

```
alumno = {
    "nombre": "Ana",
    "edad": 16,
    "curso": "IA"
}

for clave in alumno:
    print(clave)
```

```
for clave, valor in alumno.items():  
    print(clave, ":", valor)
```

Forma	Qué recorre
for clave in diccionario	Las claves.
diccionario.keys()	Las claves.
diccionario.values()	Los valores.
diccionario.items()	Pares clave-valor.

11. enumerate: obtener posición y valor

enumerate permite recorrer una secuencia obteniendo a la vez la posición y el valor. Es útil cuando necesitamos saber en qué índice está cada elemento.

```
nombres = ["Ana", "Luis", "Marta"]  
  
for posicion, nombre in enumerate(nombres):  
    print(posicion, nombre)
```

La salida será 0 Ana, 1 Luis y 2 Marta. Si queremos empezar a contar en 1, podemos indicarlo.

```
for posicion, nombre in enumerate(nombres, start=1):  
    print(posicion, nombre)
```

12. zip: recorrer varias secuencias a la vez

zip permite recorrer dos o más secuencias al mismo tiempo. Es útil cuando dos listas están relacionadas por posición.

```
productos = ["ratón", "teclado", "monitor"]  
precios = [15, 25, 120]  
  
for producto, precio in zip(productos, precios):  
    print(producto, precio)
```

En este ejemplo, ratón se asocia con 15, teclado con 25 y monitor con 120. Si las listas tienen tamaños diferentes, zip se detiene cuando termina la más corta.

zip ayuda a evitar errores de índices cuando varias listas representan datos relacionados.

13. break, continue y pass

break, continue y pass son palabras reservadas que ayudan a controlar o completar bloques de código. break termina el bucle por completo. continue salta a la siguiente vuelta. pass no hace nada: se usa como marcador cuando Python exige que exista un bloque de instrucciones, pero todavía no queremos escribirlo.

Ejemplo de `pass` como marcador temporal cuando todavía no queremos escribir el contenido del bucle:

```
for numero in range(1, 6):  
    # Más adelante escribiremos aquí el código.  
    pass
```

```
numeros = [3, 7, 10, 15]  
  
for numero in numeros:  
    if numero == 10:  
        break  
    print(numero)
```

```
numeros = [1, 2, 3, 4, 5]  
  
for numero in numeros:  
    if numero == 3:  
        continue  
    print(numero)
```

14. Bucles anidados

Un bucle anidado es un bucle dentro de otro. Se usa cuando queremos trabajar con estructuras de dos niveles, como una tabla, una matriz o una lista de listas.

```
tabla = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
for fila in tabla:  
    for numero in fila:  
        print(numero)
```

Primero se recorre cada fila. Dentro de cada fila, se recorre cada número. Hay que prestar mucha atención a la indentación.

Los bucles anidados pueden ser muy útiles, pero conviene usarlos con claridad porque aumentan la complejidad del código.

15. else en bucles

Python permite añadir `else` a un bucle. En un `for` o `while`, el bloque `else` se ejecuta si el bucle termina normalmente, sin haber sido interrumpido por `break`.

```
numeros = [1, 3, 5, 7]  
  
for numero in numeros:  
    if numero % 2 == 0:  
        print("Hay un número par")  
        break  
else:  
    print("No hay números pares")
```

Este recurso existe en Python, pero al principio puede resultar poco intuitivo. Conviene presentarlo como una ampliación, no como lo más importante de los bucles.

16. Existen otros tipos de bucles en Python

Como estructuras principales de bucle, Python tiene while y for. No existe un do while clásico como en otros lenguajes. Tampoco existe un foreach separado: el for de Python ya cumple esa función al recorrer directamente los elementos de un iterable.

Forma	Es un bucle principal	Comentario
while	Sí	Repite mientras una condición sea verdadera.
for	Sí	Recorre elementos de un iterable.
do while	No	Python no tiene esta estructura como palabra clave.
foreach	No separado	El for de Python funciona como recorrido de elementos.
Comprensiones	No como bloque	Son formas compactas de crear listas, conjuntos o diccionarios.

```
# Comprensión de listas: forma compacta
cuadrados = [numero * numero for numero in range(1, 6)]

print(cuadrados)
```

Para empezar, lo importante es dominar while y for. Las comprensiones son útiles, pero conviene verlas después de entender bien los bucles normales.

17. Errores frecuentes

Error	Ejemplo	Cómo evitarlo
Olvidar los dos puntos	for x in lista	Escribir for x in lista:
Indentar mal	print fuera del bloque	Alinear las instrucciones del bloque.
Bucle infinito	while contador < 5 sin cambiar contador	Actualizar la variable de control.
Confundir índice y valor	usar nota como si fuera posición	Nombrar bien las variables.
Pensar que range incluye el final	range(1, 5)	Recordar que llega hasta 4.

```
contador = 1

while contador <= 5:
    print(contador)
    # Si olvidamos esta línea, el bucle no termina.
    contador = contador + 1
```